resumer

Développement de logiciels embarqués

USTHB · Ingénieur 5 · Software Engineering · S9

Définitions · Comparaisons · Méthodes · Révision active

Synthèse des cours 1 à 3, du complément STM32 et du TP1 disponibles au 7 novembre 2026. Enseignant : Dr ABADA Lyes. Les supports des cours 2 et 3 conservent « Chapter 1 » dans leurs en-têtes : on suit ici les intitulés COURSE 1, COURSE 2 et COURSE 3 pour organiser les révisions.

Mise à jour : 11 novembre 2026

Sommaire

Cours 1 — Systèmes embarqués, architecture et contraintes . . . 3

Cours 1 — Temps réel, acquisition et action . . . 4

Cours 1 — Périphériques et communications . . . 4

Cours 2 — ARM, Cortex et registres . . . 5

Cours 2 — Exécution, interruptions et démarrage . . . 6

Cours 2 — Chaîne de développement et outils . . . 7

Cours 3 — GPIO, entrées et configuration électrique . . . 7

Cours 3 — Acquisition analogique, ADC et DAC . . . 8

Cours 3 — Actionneurs, PWM et temporisation . . . 9

Cours 3 — Boucle de contrôle et hystérésis . . . 10

TP1 — STM32, LED, ST-LINK et breadboard . . . 10

TP2 — Bouton, pull-up/pull-down et anti-rebond . . . 11

Révision active et sources . . . 11

Cours 1 — Systèmes embarqués, architecture et contraintes

Définition et exemples

Un **système embarqué** est un système informatique intégré à un produit pour réaliser une ou plusieurs fonctions déterminées. Il associe matériel, logiciel, alimentation et, selon le besoin, capteurs, actionneurs et interfaces de communication.

Exemples : commande d'un moteur, lave-linge, robot, système de navigation, dispositif de mesure ou objet connecté. Un ordinateur généraliste accueille de nombreuses applications ; un système embarqué est d'abord conçu autour de fonctions et de contraintes du produit. La frontière n'est pas absolue : certains systèmes embarqués exécutent Linux et des applications complexes.

Chaîne fonctionnelle : **mesurer → traiter → décider → agir**, avec communication et stockage éventuels. Tout système embarqué n'est pas connecté à Internet, ni nécessairement soumis à du temps réel dur.

MCU, MPU et SoC

Élément	Caractérisation	Exemple du cours
MCU, microcontrôleur	CPU, mémoires et périphériques largement intégrés	STM32F411
MPU, microprocesseur	Calcul central nécessitant souvent davantage de composants externes	Processeur Cortex-A dans un système complexe
SoC	Intégration de plusieurs fonctions d'un système sur une puce	Calcul, interfaces et accélérateurs selon la puce

La distinction est fonctionnelle et dépend du composant. Le MCU est adapté au contrôle, à l'acquisition et à la basse consommation ; un MPU vise souvent des besoins logiciels et mémoires plus importants.

Contraintes et qualité

Contrainte	Conséquence logicielle
RAM et Flash limitées	Suivre allocations, taille du programme et pile
CPU limité	Maîtriser le temps de traitement
Énergie	Veille, réveils planifiés et communications économes
Taille et coût	Compromis sur composants, interfaces et performances
Fiabilité	Détection d'erreur, surveillance, reprise et état sûr
Délai	Produire le bon résultat dans le temps autorisé

Un **watchdog** déclenche une action de récupération si le logiciel ne le rafraîchit plus comme prévu. Il aide à détecter un blocage ; il ne prouve pas la correction des calculs. Le duty cycling alterne activité et sommeil pour économiser l'énergie.

Cours 1 — Temps réel, acquisition et action

Temps réel dur et souple

En temps réel, la correction dépend du résultat et de son instant de production. « Rapide en moyenne » ne suffit pas à garantir une échéance.

• **Hard real-time** : manquer une échéance constitue une défaillance du système considéré.

• **Soft real-time** : un retard dégrade la qualité sans être nécessairement une défaillance totale.

Exemple du cours : période et échéance de 10 ms, calcul de 3 ms. L'exécution peut respecter ce budget ; une durée de 12 ms dépasse l'échéance. Pour un système réel, il faut aussi compter attente, interruptions et autres tâches : ce simple exemple n'est pas une preuve d'ordonnançabilité générale.

Capteur et actionneur

Un capteur transforme une grandeur physique en signal mesurable ; un actionneur transforme une commande en effet physique. Une boucle de contrôle acquiert la mesure, la compare au besoin et ajuste l'action.

```
Grandeur physique → conditionnement → ADC ou interface numérique
                  → microcontrôleur → traitement → commande → actionneur
```

Un capteur analogique demande souvent une conversion ADC. Un capteur numérique peut convertir en interne puis transmettre par I²C ou SPI. La nature de son interface ne change pas son rôle de mesure.

Échantillonnage

```
fréquence_echantillonnage = 1 / période_echantillonnage
10 ms = 0,010 s → f = 100 Hz → 100 mesures par seconde
```

Augmenter la fréquence augmente les traitements et données à gérer ; choisir selon le phénomène et les contraintes. Le support introduit l'échantillonnage sans développer ici une théorie complète de reconstruction des signaux.

Cours 1 — Périphériques et communications

Périphérique	Fonction
GPIO	Lire ou écrire des signaux numériques
Timer	Mesurer le temps ou produire des événements
ADC	Convertir un signal analogique en valeur numérique
PWM	Produire une commande modulée, par exemple pour luminosité ou moteur
Contrôleur d'interruptions	Organiser les événements et leurs priorités

Comparaison des liaisons

Interface	Principe	Usage / vigilance
UART	Liaison série asynchrone, TX/RX	GPS, console, modules ; paramètres de communication compatibles

Interface	Principe	Usage / vigilance
SPI	Horloge SCK, MOSI, MISO et sélection CS	Capteurs, mémoire, radio ; davantage de fils et sélection des périphériques
I²C	Bus SDA/SCL, appareils adressés	Plusieurs capteurs sur un bus ; vérifier adresses et configuration électrique
CAN	Bus multi-nœud à messages	Automobile et industrie, robustesse et gestion d'erreurs

Les nombres de fils du tableau du cours sont des simplifications : compter aussi la masse et les sélections nécessaires. UART et SPI peuvent être full duplex ; I²C utilise une ligne de données partagée.

Un GPS reçoit des informations de positionnement ; le logiciel lit, valide et extrait les données, par exemple depuis des phrases NMEA. Un module cellulaire permet des échanges sur le réseau mobile. **Se localiser et transmettre une position sont deux fonctions distinctes.**

Logiciel embarqué

Il peut inclure code de démarrage, pilotes, abstraction matérielle, gestionnaires d'interruptions, logique métier et protocoles. Un système bare metal peut se passer d'OS ; un RTOS gère notamment les tâches ; un système plus puissant peut exécuter un Linux embarqué. Toutes les couches présentées dans une architecture ne sont pas obligatoires.

Cours 2 — ARM, Cortex et registres

Du cœur à la carte

Il faut distinguer la conception du cœur ARM, son intégration dans une puce par un fabricant et la carte de développement qui associe cette puce à une alimentation, des connecteurs et d'autres composants. Le STM32F411 intègre un cœur Cortex-M4.

Famille / élément	Rôle principal dans le cours
Cortex-M	Microcontrôleurs, événements, contrôle, bare metal ou RTOS
Cortex-R	Applications temps réel et exigences de fiabilité, dans la classification simplifiée du support
Cortex-A	Applications complexes et OS comme Linux ou Android
ALU	Opérations arithmétiques et logiques
Registres du CPU	Données, adresses, état et contrôle de l'exécution

Ne pas confondre famille de cœur, référence du MCU et nom commercial d'une carte. La liste d'acteurs industriels de la diapositive 7 ne constitue pas une chaîne de fabrication précise de chaque carte.

Registres à mémoriser

Registre	Fonction
R0 à R12	Registres généraux : valeurs, arguments, adresses et résultats
R13 / SP	Pointeur de pile

Registre	Fonction
R14 / LR	Informations de retour d'appel
R15 / PC	Contrôle de la position d'exécution du programme
Registre d'état	Indicateurs et état du processeur

La pile sert notamment aux appels, données temporaires et sauvegardes de contexte. Une pile trop petite peut provoquer une corruption ou une panne. Parmi les indicateurs arithmétiques : zéro, signe négatif, retenue et dépassement ; ils interviennent dans les comparaisons et branchements.

Les registres CPU sont à distinguer des **registres de périphériques**. Par exemple, MODER configure un mode GPIO, IDR lit un état d'entrée et ODR décrit un état de sortie. Les périphériques sont accessibles par des adresses mémoire dédiées : c'est l'accès mémoire mappé aux E/S.

Flash et RAM

La Flash conserve notamment le programme hors tension ; la RAM contient les données d'exécution et perd son contenu à l'arrêt. Certaines zones d'adressage correspondent aux périphériques plutôt qu'à de la RAM. Lire ou écrire une adresse peut donc piloter du matériel.

Cours 2 — Exécution, interruptions et démarrage

Cycle et pipeline

Le cycle simplifié est **fetch → decode → execute**. Un pipeline fait progresser plusieurs instructions à des étapes différentes ; il améliore le débit sans garantir que chaque instruction individuelle soit plus courte. Les adresses incrémentées de quatre octets du support sont illustratives : il ne faut pas en déduire que toute instruction Cortex-M occupe toujours quatre octets.

Polling ou interruption

Polling	Interruption
Le programme vérifie régulièrement une condition	Le matériel signale un événement
Simple à suivre dans une petite boucle	Nécessite gestionnaire, priorités et partage correct des données
Le délai dépend de la fréquence des vérifications	Le délai dépend du masquage, des priorités et du traitement en cours

Déroulement : événement → sauvegarde du contexte nécessaire → gestionnaire ISR → retour à l'exécution. Une ISR courte peut signaler un travail à effectuer ensuite. Une interruption ne supprime pas tout risque de retard.

La **table des vecteurs** contient notamment le SP initial et l'entrée du gestionnaire Reset, puis les entrées d'exceptions et d'interruptions. Au démarrage, le processeur utilise ces informations pour commencer l'exécution correctement.

Événement	Idée essentielle
Reset	Démarrage ou redémarrage
NMI	Interruption non masquable par les mécanismes ordinaires
HardFault	Défaut sérieux du processeur

Événement	Idée essentielle
SysTick	Événement périodique d'un timer système
IRQ périphérique	Événement d'un timer, UART, GPIO, etc.

Les types de faute et détails disponibles dépendent du cœur. Une interruption prioritaire peut préempter une autre selon les règles configurées. Un timer peut déclencher une acquisition toutes les 10 ms, mais le respect de cette période doit être vérifié au niveau système.

Cours 2 — Chaîne de développement et outils

```
Source C → prétraitement → compilation → objets → édition de liens
         → image exécutable → HEX/BIN → programmation Flash → exécution
```

Le compilateur et l'éditeur de liens ont des rôles différents. Les bibliothèques HAL simplifient l'accès aux périphériques ; CMSIS fournit des interfaces standardisées dans l'écosystème Cortex. Le code applicatif peut employer une bibliothèque ou manipuler des registres selon le niveau d'abstraction choisi.

Le débogage utilise traces série, LED, points d'arrêt, observation des variables et registres, analyseur logique ou oscilloscope. **ST-LINK** est une sonde ; **SWD/JTAG** sont des interfaces de communication/debug. Les confondre empêche de comprendre la chaîne outil → sonde → cible.

Le complément de deux pages présente l'ajout du support STM32 dans l'IDE Arduino : installer le paquet de cartes, choisir la carte et vérifier sa présence. Ce n'est pas un chapitre supplémentaire d'architecture. Le lien du document semble avoir perdu des caractères dans sa mise en forme ; utiliser l'URL publiée officiellement par le projet pour une installation effective.

Cours 3 — GPIO, entrées et configuration électrique

Du processeur au monde physique

Le CPU traite des nombres et des instructions ; GPIO, ADC et bus relient ces traitements aux capteurs et actionneurs. La chaîne complète est : grandeur physique → capteur → signal/interface → MCU → décision → commande → driver → actionneur. Exemple du cours : mesurer une température puis commander un ventilateur.

Un signal numérique présente des états logiques discrets LOW/HIGH. Pour un système à 3,3 V, le support illustre environ 0 V et 3,3 V ; les seuils électriques effectifs dépendent du composant. **Bouton pressé ne signifie pas nécessairement HIGH** : cela dépend du câblage.

Entrée flottante, pull-up et pull-down

Une entrée non reliée à un potentiel défini peut capter du bruit et changer d'état sans action réelle. Une résistance de rappel impose un niveau au repos.

Montage du bouton	Bouton relâché	Bouton pressé	Logique
Pull-up et bouton vers GND	HIGH	LOW	Active à l'état bas
Pull-down et bouton vers l'alimentation	LOW	HIGH	Active à l'état haut

Des résistances internes peuvent être configurables en logiciel. Leur disponibilité et leur valeur dépendent de la broche et du MCU. Une résistance de rappel définit le repos ; elle ne supprime pas à elle seule les rebonds mécaniques.

Paramètres d'une broche

Paramètre	Question à résoudre
Mode	Entrée, sortie, fonction alternative ou analogique ?
Pull	Aucun rappel, pull-up ou pull-down ?
Type de sortie	Push-pull ou open-drain ?
Vitesse de sortie	Quels fronts/configuration électrique sont nécessaires ?

Une sortie **push-pull** commande activement le niveau haut et le niveau bas. Une sortie **open-drain** impose le niveau bas ou se met en haute impédance ; un pull-up est normalement nécessaire pour obtenir le niveau haut. Le transistor coupé ne produit donc pas activement un HIGH. Ne pas confondre le type de sortie, le rappel d'entrée et la fréquence d'un signal PWM.

Rebond et anti-rebond

Un seul appui mécanique peut produire plusieurs transitions électriques. Sans filtrage, un compteur peut incrémenter plusieurs fois pour un appui.

Le support propose de relire l'entrée après un délai illustratif de 10 à 50 ms. Une meilleure stratégie mémorise le changement, démarre une temporisation, puis confirme seulement un état resté stable suffisamment longtemps. Ce filtrage peut fonctionner sans immobiliser toute la boucle. Le délai doit être choisi selon le bouton et l'application.

Cours 3 — Acquisition analogique, ADC et DAC

Types d'interface

Un capteur numérique peut fournir un état binaire ou une mesure codée via I²C, SPI, UART ou un autre bus. Un capteur analogique fournit une tension variable, généralement convertie par un ADC. La conversion peut aussi se faire dans le capteur lui-même avant transmission numérique.

Conversion	Sens	Usage
ADC	Tension analogique → nombre	Acquisition d'une mesure
DAC	Nombre → tension analogique	Génération de signal ou commande analogique

Nombre de codes et conversion approximative

```
ADC sur N bits : 2^N codes, de 0 à 2^N − 1
Code ≈ (Vin / Vref) × (2^N − 1)
Vin ≈ Code × Vref / (2^N − 1)
Pas idéal de quantification ≈ Vref / 2^N
```

La formule du code est l'approximation par les extrémités utilisée dans le cours. Un ADC réel quantifie selon sa fonction de transfert ; la valeur entière dépend des seuils et de la convention d'arrondi. Pour 12 bits et Vref = 3,3 V, les codes sont 0 à 4095 et le pas idéal vaut environ **0,806 mV**.

Vin	Calcul approché du support	Code arrondi pour l'exercice
1,65 V	1,65 / 3,3 × 4095 = 2047,5	Environ 2048
1,70 V	1,70 / 3,3 × 4095 ≈ 2109,55	2110

Vin	Calcul approché du support	Code arrondi pour l'exercice
1,80 V	1,80 / 3,3 × 4095 ≈ 2233,64	2234
2,20 V	2,20 / 3,3 × 4095 = 2730	2730

Résolution et précision diffèrent. Davantage de bits donne des niveaux plus fins ; l'erreur finale dépend aussi du capteur, de la référence, du bruit, du câblage et de la calibration. Une formule idéale ne garantit pas la précision réelle du montage.

Cours 3 — Actionneurs, PWM et temporisation

Driver et commande de puissance

Un GPIO ne fournit qu'un courant et une tension limités. Pour un moteur ou une charge nécessitant davantage de puissance, la commande passe par un driver, transistor, relais ou ESC adapté à la charge. Le GPIO transmet la consigne ; l'alimentation de puissance fournit l'énergie.

PWM et rapport cyclique

Une PWM commute rapidement entre LOW et HIGH. Son rapport cyclique décrit la fraction de la période passée au niveau haut.

```
D = temps_HIGH / période
D_pourcentage = 100 × temps_HIGH / période
f = 1 / période
```

Exemple : période de 2 ms et temps haut de 0,5 ms → D = 25 % et f = 500 Hz. À période identique, 25 %, 50 % et 90 % correspondent à des durées hautes différentes. La PWM ne transforme pas directement la broche en une source analogique intermédiaire ; la charge et éventuellement un filtrage déterminent l'effet obtenu. Applications du cours : luminosité, commande moteur, chauffage et commande adaptée de servomoteur.

Timers, PSC et ARR

Un timer matériel compte des impulsions pour mesurer des durées, produire des événements, de la PWM ou des timeouts. Avec une horloge de 1 MHz, un tick sans division dure 1 microseconde ; 1000 ticks représentent 1 ms.

Dans le mode simple de comptage croissant présenté pour STM32 :

```
f_compteur = f_timer / (PSC + 1)
T_evenement = (PSC + 1) × (ARR + 1) / f_timer
f_evenement = f_timer / ((PSC + 1) × (ARR + 1))
```

Le compteur parcourt 0 à ARR inclus : les **+1** sont indispensables. Pour f_timer = 1 MHz, PSC = 0 et ARR = 999 donnent T = 1 ms. Avec PSC = 9, f_compteur = 100 kHz : chaque tick dure 10 microsecondes. Ces calculs partent de l'horloge du timer indiquée ; il faut vérifier sa configuration effective sur la carte.

Temporisation bloquante ou non bloquante

`HAL_Delay(1000)` bloque le flot d'exécution appelant pendant l'attente ; les autres actions de la même boucle sont retardées. Cela ne signifie pas que toutes les interruptions du processeur sont nécessairement arrêtées.

Pour faire avancer la boucle, mémoriser un instant et comparer le temps écoulé :

```
uint32_t now = HAL_GetTick();
if ((uint32_t)(now - last) >= 1000U) {
    last = now;
    toggle_led();
}
```

`last` doit être un compteur non signé de même type, initialisé avant la boucle. Cet exemple illustre une action périodique sans attente bloquante. La boucle peut encore lire un capteur et traiter des communications entre deux actions. Le respect d'une échéance dépend toujours du temps des autres traitements.

Cours 3 — Boucle de contrôle et hystérésis

La boucle de base lit les capteurs, traite les données, commande les actionneurs, communique puis recommence. Avec un ventilateur, le seuil simple « température > 30 °C » peut provoquer des commutations fréquentes lorsque la mesure oscille autour de 30 °C.

L'**hystérésis** emploie deux seuils :

Température	Décision
Supérieure à 30 °C	Ventilateur ON
Inférieure à 28 °C	Ventilateur OFF
Entre 28 et 30 °C, bornes comprises pour ces gardes strictes	Conserver l'état précédent

La mémoire de l'état est essentielle ; l'état initial doit être défini. Le système évite ainsi de basculer à chaque petite variation autour d'un unique seuil. L'anti-rebond stabilise l'interprétation d'un contact ; l'hystérésis stabilise une décision à seuil : les deux répondent à des situations différentes.

TP1 — STM32, LED, ST-LINK et breadboard

Chaîne d'outils et téléversement

Le TP présente l'IDE Arduino avec le support des cartes STM32, STM32CubeProgrammer et une sonde ST-LINK V2. La carte et le mode de téléversement doivent correspondre au matériel employé. La liaison SWD repose notamment sur SWDIO, SWCLK, une masse commune et les connexions d'alimentation/référence prévues par le montage.

Il faut distinguer programmation du microcontrôleur, alimentation de la carte et liaison série. Le moniteur série doit utiliser le débit configuré par le programme, ici **115200** ; un câble USB ne garantit pas à lui seul que la sortie série choisie est correctement configurée.

LED active à LOW

Dans la carte décrite par le TP, la LED intégrée sur **PC13** s'allume avec LOW et s'éteint avec HIGH. Il s'agit d'une propriété de ce câblage, pas d'une règle universelle des STM32.

Montage	Chemin du courant quand la LED est allumée	Niveau actif
Sinking	Alimentation → résistance → LED → GPIO	LOW
Sourcing	GPIO → LED → résistance → GND	HIGH

Dans les deux cas, la résistance série limite le courant. Le niveau logique nécessaire dépend du montage ; il faut respecter les caractéristiques électriques de la broche et de la LED.

Programme et période de clignotement

`setup()` initialise la broche en OUTPUT et la communication série ; `loop()` répète les actions. LOW pendant 500 ms puis HIGH pendant 500 ms donnent une période totale de **1 s**, soit **1 Hz** pour le cycle complet. Le TP utilise une attente bloquante ; le cours 3 montre comment l'éviter lorsqu'il faut aussi effectuer d'autres tâches.

Une attente `while (!Serial)` peut empêcher d'atteindre `loop()` si la liaison attendue n'est pas disponible. Si la LED ne démarre pas, vérifier cette attente, la sélection de carte, la broche et le téléversement avant de conclure à un défaut de LED.

Breadboard

Les groupes de contacts internes assurent les connexions sans soudure. Le séparateur central isole les deux côtés ; les rails d'alimentation sont distincts des groupes de signaux et peuvent être coupés au milieu selon le modèle. Vérifier les continuités réelles plutôt que supposer que toute une ligne est reliée.

TP2 — Bouton, pull-up/pull-down et anti-rebond

Résistance de tirage

Montage	Câblage	État actif
Pull-up (`INPUT_PULLUP`)	Résistance vers VCC, bouton vers GND	LOW
Pull-down (`INPUT_PULLDOWN`)	Résistance vers GND, bouton vers VCC	HIGH

Les broches du STM32F411 disposent de résistances internes dans les deux sens. Le bouton intégré de la carte est relié à GND : il fonctionne en pull-up mais pas en pull-down.

Détection de front et anti-rebond

Le programme bascule la LED PC13 sur un **front** : passage HIGH → LOW en pull-up (LOW → HIGH en pull-down), en comparant l'état courant à `lastButtonState`. Un `delay(200)` absorbe le rebond mécanique du contact.

```
bool current = digitalRead(BUTTON_PIN);
if (lastButtonState == HIGH && current == LOW) {
  ledState = !ledState;
  digitalWrite(LED_PIN, ledState);   // LOW = LED allumée
}
lastButtonState = current;
delay(200);
```

Sans bouton externe, un fil relié à A0 simule l'appui : vers GND en pull-up, vers 3,3 V en pull-down. Les exemples de code fournis par l'enseignant sont rangés dans `TP/code/` (LED, bouton, saisie série, lecture analogique, PWM pour ESC).

Révision active et sources

1. **Pourquoi un résultat exact peut-il rester incorrect ?** Il arrive après l'échéance d'un système temps réel.

2. **Qui mesure, qui agit ?** Le capteur mesure ; l'actionneur produit l'effet physique.

3. **Que vaut la fréquence pour une période de 5 ms ?** 200 Hz.

4. **SP et PC sont-ils interchangeables ?** Non : pile et exécution sont deux fonctions distinctes.

5. **Une interruption garantit-elle une réaction immédiate ?** Non ; analyser masquage, priorités et latence.

6. **Pourquoi une écriture mémoire peut-elle allumer une LED ?** L'adresse peut désigner un registre GPIO.

7. **Quel intérêt du pipeline ?** Augmenter le débit de traitement d'instructions.

8. **Un pull-up donne quel niveau lors d'un appui vers GND ?** LOW, donc logique active à l'état bas.

9. **Un open-drain produit-il activement HIGH ?** Non, il libère la ligne ; le pull-up fournit le niveau haut.

10. **Combien de codes pour 12 bits ?** 4096, numérotés 0 à 4095.

11. **Pourquoi ajouter 1 à PSC et ARR ?** Le diviseur et le comptage représentés incluent zéro dans le mode étudié.

12. **Pourquoi utiliser deux seuils pour le ventilateur ?** Conserver l'état dans une bande intermédiaire et éviter les commutations répétées.

13. **Deux attentes de 500 ms donnent quelle fréquence de clignotement ?** Un cycle de 1 s, donc 1 Hz.

Sources : `COURS/Course1-Introduction-Embedded-Systems.pdf` (49 pages), `COURS/Course2-ARM-Architecture.pdf` (46 pages), `COURS/Course2-STM32-Extension-Setup.pdf` (2 pages), `COURS/Course3-Embedded-IO.pdf` (55 pages : GPIO p. 9–24, acquisition p. 25–31, actionneurs/PWM p. 32–36, timers p. 37–45, contrôle p. 46–55) , `TP/TP1.pdf` (8 pages) et `TP/TP2.pdf` (4 pages). Les formules présentes dans les schémas des pages 40–41 ont été consultées visuellement. Linux embarqué détaillé, synchronisation avancée et méthodes de conception restent des thèmes annoncés sans support dédié présent.